LocoManiLab: A Unified Testbed for Data, Policies, Controllers, and Deployment in Humanoid Loco-Manipulation

Dewei Wang*, Jinrui Han*, Jiyuan Shi, Chenyun Zhang, Xianbo Lu, Jiabao Wu, Jiqin Feng, Jipeng Kong, Chenjia Bai, and Xuelong Li, *Fellow, IEEE*

***Abstract*—Benchmarking humanoid loco-manipulation is challenging because performance depends on both high-level policies and low-level whole-body controllers. Fair comparison requires a platform that supports diverse policies and controllers, covers varied tasks and datasets, and provides a unified, reproducible evaluation pipeline with standardized metrics. To meet these requirements, we introduce LocoManiLab, a unified testbed for humanoid loco-manipulation. On the task and evaluation side, LocoManiLab spans five household scene categories with more than 100 atomic and more than 100 compositional tasks, scored by task graphs that report success, progress, and the stage at which a rollout fails. At the same time, LocoManiLab provides a unified interface that decouples policies from controllers, connecting a target roster of twelve policies to five whole-body controllers across multiple action interfaces, and extends readily to new ones. Teleoperated demonstrations comprise approximately 50 hours of data across more than 10 scene instances, complemented by a data-synthesis pipeline that fine-tunes a motion generation model against a frozen controller to generate new data. Using LocoManiLab, we comprehensively evaluate mainstream policies and whole-body controllers, covering training data scale, generated data, and task randomization across the full pipeline. The experimental program examines policy generalization and randomization robustness, multi-controller execution, data scale, generated demonstrations, and paired simulation and hardware performance. [Final results and their conclusions are pending.]**

***Index Terms*—Benchmarking, humanoid robots, imitation learning, loco-manipulation, vision-language-action models, whole-body control.**

I. Introduction

HUMANOID robots can navigate human spaces, change posture, and manipulate across a workspace far larger than a fixed arm can reach. Most approaches to these behaviors are hierarchical, where a high-level *policy*, typically a vision-language-action (VLA) or world-action model (WAM), consumes images and a language instruction and emits an intermediate command such as a base velocity, an end-effector target, or a reference motion [1]–[4], and a low-level *controller* runs at a much higher rate and realizes that command as joint-level actions under balance, contact, and actuator constraints [5]–[8]. An evaluated behavior is therefore produced by a specific pairing of policy and controller rather than by the policy alone.

Controllers differ in the command they accept, from a base velocity with end-effector targets to a whole-body reference motion to motion tokens, so the same instruction is executed under a different feasibility envelope and a policy that succeeds with one tracker can fail with another. A binary success rate hides this, recording that the task ended rather than whether the robot reached the goal with more mechanical work, jerkier joint motion, or a fall that came after the goal predicate was already satisfied, so attribution needs the controller varied explicitly and execution scored alongside completion. Existing work rarely does either, because progress has come along separate axes, each advanced while the rest stay fixed. Simulation suites scale tasks and scenes while folding the controller into the environment [9], [10], data-generation pipelines produce demonstrations but validate them with one policy of their own design [11], [12], and hardware studies vary the policy while executing every variant through one tracker [3]. Where the controller does vary, comparison stays inside a single action interface on a handful of simulated tasks [13], so a reported number describes a pairing rather than a policy and cannot say which half earned the score.

Coverage is thin along exactly the axes a benchmark is meant to establish. Reported policy panels are small and architecturally homogeneous, so different VLA and WAM families are seldom compared under identical tasks and data [9], [10]. Task suites run to a few dozen entries rather than hundreds, and the demonstrations released alongside them stay comparatively small [3], [13]. Whole-body data generation is demonstrated at limited volume, leaving both large-scale synthesis and the actual contribution of generated data unexamined [11], [12]. Evaluation protocols are thin in the same way. Scores are reported on the scenes and object configurations a policy was trained on, so a table registers a capable stack and a

*D. Wang and J. Han contributed equally to this work.

D. Wang and J. Feng are with the University of Science and Technology of China, China, and also with the Institute of Artificial Intelligence (TeleAI), China Telecom, China (e-mail: dweik0607@gmail.com; smgj@mail.ustc.edu.cn).

J. Han is with the University of Hong Kong, Hong Kong, and also with the Institute of Artificial Intelligence (TeleAI), China Telecom, China (e-mail: jinrui.han@connect.hku.hk).

X. Lu is with Zhejiang University, China, and also with the Institute of Artificial Intelligence (TeleAI), China Telecom, China (e-mail: booerlu@gmail.com).

J. Wu is with Shanghai Jiao Tong University, China (e-mail: xskxsjwjz@sjtu.edu.cn).

J. Kong is with ShanghaiTech University, China (e-mail: kongjp2024@shanghaitech.edu.cn).

J. Shi, C. Zhang, C. Bai, and X. Li are with the Institute of Artificial Intelligence (TeleAI), China Telecom, China (e-mail: shijy15@chinatelecom.cn; zhangcy60@chinatelecom.cn; baicj@chinatelecom.cn; xuelong_li@ieee.org).

Corresponding author(s): Chenjia Bai and Xuelong Li.



Fig. 1. LocoManiLab evaluates a humanoid loco-manipulation stack as a whole rather than a policy in isolation. Five household scene categories supply 103 atomic and 132 compositional tasks (1). Teleoperated demonstrations are paired with a matched corpus from a fine-tuned motion diffusion model whose references a frozen controller executes and filters (2). Twelve policies and five whole-body controllers are registered behind a common interface, so a policy runs with every controller that accepts the command type it emits (3). Each pairing is scored on seen and held-out splits with task-level and motion-level metrics, and the platform, the data, and the leaderboard are released (4).

memorizing one at the same value, and the held-out task definitions and scene instances that would separate them are seldom defined as part of the suite [9], [13].

We introduce LocoManiLab, shown in Fig. 1, a full-stack platform for humanoid loco-manipulation with four main features:

Full-stack pipeline: Simulation and hardware share the same pipeline. Data collection and processing, policy training, the handoff to each controller, and policy inference all execute inside one framework. That keeps a result reproducible from its recorded configuration and lets a new policy, controller, or scene be added without rebuilding the stack.

Extensible policy and controller panel: LocoManiLab supports and evaluates 12 policies and 5 whole-body controllers. Policies span VLA and WAM families and are registered as plugins that declare their own configurations. Controllers cover three action interfaces, and each policy can be benchmarked with every controller that accepts the interface it produces.

Task and dataset: The task suite is organized by capability and includes 103 atomic and 132 compositional tasks over five household scene categories. Data comes from human teleoperation, which has collected 51.5 hours, and from generation, which targets a further 50 hours. Teleoperation drives any of the 5 controllers from a VR interface, so a demonstration is collected on the backend a policy is later evaluated on. Generation pairs a fine-tuned diffusion motion model with a controller that executes its references in the scene and filters the rollouts it produces.

Unified evaluation and open release: LocoManiLab evaluates policies and controllers in simulation across atomic tasks, compositional tasks, generated data, unseen scenes, and unseen tasks, reporting where each policy and each controller is strong or weak rather than a single aggregate score. We release the platform and the data, together with a leaderboard for evaluating new models.

Taken together, these features make LocoManiLab a unified testbed for data, policies, controllers, and deployment in humanoid loco-manipulation. LocoManiLab supplies a common ground for humanoid loco-manipulation, one on which policies of different families and controllers of different designs are supported, deployed, and evaluated. The platform and the data are released so that a new component can be measured and more experiments can be conducted. The experimental program evaluates the full policy panel on seen and held-out conditions and under domain randomization, compares controllers and data scales with a VLA and a WAM, studies generated data, and pairs simulation with hardware on three tasks. The completed results will be used to identify which parts of the pipeline limit humanoid loco-manipulation. [Summarize the three headline findings once the studies are rerun: seen versus unseen behavior, execution quality across controllers, and what each data source contributes.]

II. Related Work

A. Humanoid Whole-Body Controllers

Whole-body controllers give a humanoid the low-level competence a high-level policy assumes, and three lines dominate the recent literature. One tracks human keypoints or pose as

TABLE I
REPORTED SCOPE OF REPRESENTATIVE HUMANOID LOCO-MANIPULATION SYSTEMS.

System	Sim data (h)	Real data (h)	Sim tasks	Num. upper policy	Num. lower controllers	Data-gen.	Real world deployment.
HumanoidBench [9]	–	–	27	–	–	–	–
SIMPLE [10]	3.0	7.9	60	7	2	✓	✓
HumanoidArena [13]	5.0	–	7	1	2	–	–
OpenHLM [3]	–	–	–	3	2	–	–
LOCOMANILAB	51.5	10.0	235	12	5	✓	✓

a universal interface, which improves robustness and scales the teleoperation-to-data loop [14], [15]. A second decouples the body, distilling trajectory optimization into whole-body control [6] or training locomotion and motion imitation adversarially [7]. A third trains one general controller for diverse motions, through mixture-of-experts architectures [16], [17], multiple reference modalities [5], full-size payload-aware teleoperation [8], or a behavior foundation model scaled so that one model covers whole-body behavior rather than a fixed skill set [18], [19]. Each is released as a standalone backend that accepts its own command type, a human-like reference, a velocity command paired with an arm target, or a latent token vocabulary, and these are not interchangeable encodings of the same intent. A policy therefore cannot be moved between backends without a conversion that is seldom reported, which is why controller choice is usually settled once and left out of the comparison. LOCOMANILAB treats it as a swappable variable instead, declaring the command type each backend accepts so that an unsupported pairing is reported rather than silently bridged, and measuring execution quality across five of them.

B. Humanoid Whole-Body Loco-Manipulation

Humanoid loco-manipulation policies have moved from fixed skill repertoires learned from human motion [20]–[22] to language-conditioned models that map vision and instructions to whole-body action, either as vision-language-action models pretrained on egocentric human video [1], [2] or as world-action models that learn visual dynamics jointly with actions [4], [23]–[25], both building on general-purpose VLA architectures [26]–[30]. The shift buys generality with data, since a language-conditioned model inherits broad visual and semantic priors [31] and needs only a task-specific adaptation set, which moves the binding constraint from reward design to demonstration supply. Because teleoperated whole-body data is expensive, one response makes a single demonstration go further, recovering long-horizon bimanual behavior from one human video [32]. Another synthesizes demonstrations in simulation through whole-body planning, real-scene reconstruction, procedural scene generation, or retargeting of human references [11], [12], [33], and validates them with the policy and controller that produced them. Each work along both threads supplies one policy of its own design, so LOCOMANILAB evaluates open models from these families under a single protocol. It also puts the synthesis question to a test that the literature leaves open, comparing generated, augmented, and teleoperated episodes at matched episode count under an identical recipe in Sec. IV-E1.

C. Benchmarks for Manipulation and Loco-Manipulation

Simulation benchmarks have grown from tabletop skill suites [34]–[36] to broader simulation frameworks with diverse scenes, tasks, and demonstration datasets [37]–[39], establishing cross-factor generalization splits, common formats for pooling heterogeneous robot data, and sim-to-real ranking validity [40], [41]. Audits show that reported VLA conclusions shift with the action representation a model is given and with the environment it is evaluated in [42], motivating our paired initial conditions and held-out splits. What that line established is less a catalogue of tasks than a methodology. A benchmark is now expected to release the data its tasks were solved with, to define splits along named factors rather than one undifferentiated held-out set, and to show that a simulated ranking survives contact with hardware. The audits add the caution that a ranking is a property of a measurement protocol as much as of a model.

Humanoid benchmarks have inherited the task suite from this line but not the rest of it. HumanoidBench establishes the scope of simulated locomotion and whole-body manipulation [9] while folding the controller into the environment, so a score there is a property of the pair without saying so. SIMPLE goes furthest on the policy axis, comparing seven model families under contact-rich simulation and photorealistic rendering [10], and still holds the controller close to fixed. HumanoidArena is the closest to the question we ask and supplies the most direct evidence that it is worth asking, reporting that transfer across trackers is fragile [13], although on seven simulated tasks with a single policy and two controllers inside one action interface. The axis has therefore been identified but not swept. As Table I summarizes, no existing system combines a capability-structured task suite at scale, a controller varied as an explicit factor, and a data volume tied to released, trainable episodes.

III. LOCOMANILAB

A. Overview

LOCOMANILAB is a full-stack benchmark for humanoid loco-manipulation whose design brings together a scene library spanning five household categories, capability-oriented tasks with verifiable subgoals, a shared schema for teleoperated and generated demonstrations, a panel of policies and compatible controllers, and common task-level and motion-level metrics for simulation and hardware. Fig. 1 shows how these parts connect, and the remainder of this section describes each in turn. The unit of evaluation is a policy paired with a controller rather than a policy alone. A high-level policy π

maps egocentric observations and a language instruction to a command z_t, and a whole-body controller converts z_t to joint targets at a control rate of 50 Hz. Every task, demonstration, and controller targets one embodiment, a Unitree G1 humanoid with 29 body joints and a three-finger hand of seven joints on each arm; compatibility additionally depends on the command representation and controller checkpoint. LocoManiLab groups commands into three families, namely base commands with arm or upper-body targets, whole-body reference motion, and motion tokens, with a specific command layout for each backend. A policy–controller pair is supported when the controller accepts the policy's declared command layout, including its coordinate frames and normalization; any conversion between layouts must be explicit. Controller choice determines how a command is realized, so every result is published together with the controller, interface, and deployment settings that produced it.

Collection, training, deployment, and evaluation use a versioned episode format with backend-specific observation and command fields, together with collection and deployment metadata. The modality layout identifies each field's meaning and dimensions; object and contact state and task progress belong to the environment and evaluation layer and are not assumed to be present in every demonstration. This separation lets policies and controllers share data-loading and dispatch infrastructure while preserving their native representations, and the simulation and hardware collectors use the same modality layout for a given backend. Scenes are simulated in MuJoCo [43] and built from reusable assets whose sources are recorded with the scene specification. Objects and kitchen fixtures come from RoboCasa [37], largely derived from Objaverse [44], and room backgrounds and floor plans come from MolmoSpaces, which converts iTHOR scenes into the same simulator [45]. Office scenes are laid out with Holodeck [46], which extends the four iTHOR-derived room categories with language-guided layouts. Automated checks detect scale errors, unstable inertial parameters, interpenetration, and unreachable initial configurations.

B. Policies and Controllers

Policy plugins declare required observations, native action representation, chunk length, inference frequency, normalization, and checkpoint provenance, which accommodates VLA models conditioned on vision and language, world-action models that predict temporally extended motion, and lightweight imitation baselines. Each plugin emits z_t on its declared interface, while a common driver consumes action chunks at the controller rate and requests new chunks at a separately configured inference rate. The driver supports synchronous replanning in simulation and asynchronous inference during deployment, where buffered actions continue to execute while a request is pending and the next chunk's entry index compensates for inference delay up to a configured cap. The 50 Hz simulation control rate therefore specifies simulated time per step, not a guarantee of wall-clock real-time throughput. The intended policy panel follows the twelve-policy roster of Table III, namely StarVLA with its GR00T- and OFT-backbone variants [47], [48], PRTS [49], $\pi_{0.5}$ [26], GR00T N1.7 [50], XR-1 [51], InternVLA-A1.5 [52], Ψ_0 [1], X-VLA [29], SmolVLA [28], Diffusion Policy [53] as a from-scratch baseline, Fast-WAM [24], and DreamZero [23]. Registration, interface validation, and closed-loop evaluation are distinct stages, and Sec. IV-B reports policy–task coverage for the registered panel. Each plugin stays deliberately thin, covering the native environment, dataset conversion, training launcher, checkpoint layout, observation schema, and deployment server while retaining the upstream implementation behind a common request–response boundary, so one versioned episode supports schema validation, model-specific conversion, SFT, and open- or closed-loop evaluation without importing incompatible model stacks into a single process.

Five controller backends sit beneath this panel and execute at 50 Hz. Their supported command types differ, so each comparison records both the backend and its action interface. *AMO* combines a learned lower-body controller [6] with arm targets obtained through IK or motion retargeting during teleoperation; the IK split also appears in SIMPLE [10]. Its policy-facing command has 22 entries, namely 14 arm-joint targets, torso orientation and height, planar velocity, yaw rate, and target heading. *SONIC* tracks whole-body motion with a learned general motion tracker [5]. Teleoperation and locomotion planning supply its motion references; the policy-facing token interface instead supplies 64-dimensional motion tokens directly to the matching decoder, with token semantics tied to the encoder–decoder checkpoint. *HEFT* tracks VR-derived reference motion using a policy trained with privileged motion guidance and a payload curriculum [8]; its adapter accepts root position, root quaternion, and 29 body-joint reference angles. *ALMI* coordinates upper- and lower-body execution through locomotion and motion-imitation policies [7]. Its command combines planar velocity, yaw rate, and base height with a 34-dimensional upper-body reference containing waist orientation, two end-effector poses, and 17 joint targets. *ScaleBFM* provides a behavior foundation model for humanoid whole-body control [18] and shares HEFT's root-pose-and-joint-reference layout. All five backends additionally receive 14 finger-joint targets through a separate hand-control path. The resulting policy action dimensions are 36 for AMO, 78 for SONIC's token interface, 52 for ALMI, and 50 for HEFT and ScaleBFM. Controller adapters preserve each command's coordinate frames, joint order, and normalization; the backend and checkpoint remain part of a pairing's identity even when its command layout is shared. Evaluation rollouts additionally record the commanded reference, the measured state, an action or torque proxy, contacts, actuator saturation, and the termination reason, which are the signals the motion-level metrics of Sec. IV-A3 are computed from.

C. Task Design

The suite is organized first by scene category and second by capability, rather than as a flat list of object configurations, so that a result shows both where a stack fails and what it cannot yet do. Five household categories, namely kitchen, living room, bedroom, bathroom, and office, span 13 scene

instances, and each task is defined for one category and given as a natural-language instruction. The current suite holds 103 atomic and 132 compositional tasks. Atomic tasks isolate one skill on one object or fixture and are typed by the body resources observed in their demonstrations. *Stationary* denotes manipulation without meeting the locomotion or whole-body criteria below; *locomotion* denotes walking, turning, or stepping, and *whole-body* denotes squatting or leaning that changes the manipulation workspace. These are operational labels derived from recorded motion using the thresholds in Sec. III-D, rather than claims about the minimum motion every successful policy must use. Each atomic task also carries a skill primitive, namely navigating, opening or closing an articulated part, picking up or lifting, picking and placing, or pushing, pulling, pouring, and reorienting, and these primitives form the vocabulary from which compositional tasks are built.

Compositional tasks chain these primitives into one instructed episode, so difficulty is raised by adding skills and horizon rather than by changing object texture. They are graded by chain length, currently two to four steps, as in opening a drawer, taking out a tissue box, carrying it to the desk, and placing it there, and most chains begin by walking to a landmark, so locomotion links the manipulations. Each step is linked to the atomic task that trains the same skill on the same object whenever one exists, so a failed compositional rollout can be traced to a missing atomic capability rather than attributed to horizon alone. This holds for 80% of manipulation steps, whereas most navigation steps lead to landmarks that no atomic task covers. Every compositional task also exposes an ordered task graph whose subgoals give a curriculum for collection and online RL and yield partial progress when binary success is too sparse. A subgoal is credited only when its physical predicate and the task's safety invariants hold. Final success additionally requires the robot to remain upright and the terminal goal predicates to hold continuously for a task-specific dwell interval, fixed before evaluation and shared by all policy–controller pairs on that task, so that a transient or unsafe completion does not count. Held-out task definitions and held-out scene instances define the two generalization conditions that evaluation scores alongside the training tasks (Sec. IV).

D. Dataset

Three collection modes use the versioned episode format with backend-specific layouts, namely teleoperation on the physical humanoid, the same operator interface on the simulated robot under controlled initial states, and the motion-synthesis pipeline below. Model-specific conversion selects compatible observation and action fields for mixed-source SFT and data-source comparisons.

1) Data Collection: The primary effort is human teleoperation, with commanded motion and measured robot state recorded as distinct quantities so that the policy's target representation can be separated from controller execution. Operators use a VR headset and hand controllers that provide head and hand poses; backend-specific retargeting converts these inputs into arm targets or whole-body references. Analog hand-controller triggers are mapped to seven robot finger-joint targets per hand, while measured robot finger positions are stored in the state fields. In SONIC's planner mode, the operator also commands movement direction, speed, and pelvis height. Episodes are recorded at a nominal 50 Hz with egocentric RGB images at 320×240 and backend-specific state and command fields; metric depth is an optional sidecar. SONIC, HEFT, ALMI, and ScaleBFM record 43 measured body-and-hand joint positions, whereas AMO records 28 measured arm-and-hand joints together with the previous torso orientation and height commands in a 32-dimensional state. Measured root orientation and projected gravity are separate fields; SONIC additionally stores a 14-dimensional end-effector state computed from the joint state by forward kinematics in the pelvis frame. In simulation, image observables can precede the directly read joint state by up to one control tick; collection and simulated policy evaluation use the same sampling convention. Typed action fields distinguish SONIC's motion tokens and execution-layer joint targets from ALMI's base commands and upper-body references, and the schema version identifies each layout's semantics. Each episode also records the operator's wall-clock time to success, the scene seed, and the jitter radii applied at reset, which displace the robot by up to 5 cm and each object by up to 2 cm so that repeated demonstrations of one task are not replays of a single initial state. Quality checks reject episodes with missing synchronization, invalid object state, unsafe termination, or inconsistent annotations, while failed-but-valid attempts are retained for online learning and diagnostics.

2) Data Generation: To expand beyond teleoperation, the data-generation pathway uses an RL-trained motion generator. The motion diffusion model Kimodo [54] proposes a reference-motion chunk from text and start/goal constraints such as a root path and end-effector targets, following the text-driven motion diffusion literature [55], [56], and a frozen SONIC controller executes it in simulation. Because the controller does not track perfectly, the executed end-effector position can differ from the target even for a kinematically valid reference. We therefore fine-tune Kimodo against the frozen controller on a reward measured from the executed trajectory rather than from the proposed reference, so that it anticipates closed-loop execution bias instead of only planning kinematically. The generator produces a chunk by running K denoising steps from Gaussian noise under a condition c that carries the instruction and the start and goal constraints. For stochastic steps $k \in \mathcal{S} = \{k : \sigma_k > \sigma_{\min}\}$, the transition is

$$g_\theta\left(m^{(k-1)} \mid m^{(k)}, c\right) = \mathcal{N}\left(\mu_\theta\left(m^{(k)}, k, c\right),\ \sigma_k^2 I\right), \quad (1)$$

while the remaining steps use the transition mean deterministically and are excluded from the policy loss. The chain from $m^{(K)}$ to the proposed chunk $m^{(0)}$ forms a sampled policy trajectory parameterized by the denoiser weights θ. The chunk is resampled from the generator's 30 Hz to the controller's 50 Hz and executed from its first frame, which returns a realized trajectory and a scalar return R that credits the hand and the root for reaching their targets and holding there.

We sample $G > 1$ candidates for each condition and assign each candidate's execution return to its stochastic denoising transitions. For candidate i in group $g(i)$, the advantage is the group-normalized return

$$\hat{A}_i = \frac{R_i - \bar{R}_{g(i)}}{\max(s_{g(i)}, 10^{-6})}, \tag{2}$$

where $\bar{R}_g$ and s_g are the group mean and sample standard deviation. For each stochastic transition, we normalize the Gaussian log probability by the number D_i of elements in the sampled motion state:

$$\begin{aligned} \ell_{\theta,i,k} &= D_i^{-1} \log g_\theta(m_i^{(k-1)} \mid m_i^{(k)}, c_i), \\ \Delta\ell_{i,k} &= \ell_{\theta,i,k} - \ell_{\theta_{\text{old}},i,k}, \\ \tilde{\rho}_{i,k} &= \exp(\text{clip}(\Delta\ell_{i,k}, -10, 10)). \end{aligned} \tag{3}$$

Thus $\tilde{\rho}$ is a ratio formed from normalized log probabilities, rather than the unnormalized joint-density ratio. Neither the simulator nor the controller is differentiated through; we recompute log probabilities on stored denoising transitions and minimize the PPO-style clipped loss

$$\begin{aligned} \mathcal{L}_\pi(\theta) = -\mathbb{E}_{i,k}\big[\min\big(&\tilde{\rho}_{i,k}\hat{A}_i, \\ &\text{clip}(\tilde{\rho}_{i,k}, 1-\epsilon, 1+\epsilon)\hat{A}_i\big)\big]. \end{aligned} \tag{4}$$

The expectation averages over candidates and their stochastic steps $k \in \mathcal{S}$, and ϵ is the clipping width. A value head receives the pooled text embedding h_i and is trained through the auxiliary term $0.5\,\mathbb{E}_i[(V_\phi(h_i) - R_i)^2]$ added to $\mathcal{L}_\pi$; it does not supply the baseline in Eq. 2. Execution returns update the denoiser through these sampled transitions, while the controller and any downstream deployed policy remain frozen. Conditions seen during fine-tuning are drawn half at random and half from the task suite, which keeps the generator general while exposing it to the configurations the benchmark asks for, and we fine-tune in two modes, updating all denoiser weights or training rank-16 adapters that leave the base weights frozen. Scene-level generation is designed to couple these approach and posture motions with the inverse-kinematics arm controller cuRobo [57] for fine end-effector control during grasping, following the decoupled split of Sec. III-B. Sec. IV-E1 evaluates the resulting corpus as a source of training demonstrations. Inclusion in the generated corpus requires complete episodes that pass the same synchronization, state-validity, and annotation checks as teleoperated data, and downstream data-source comparisons are defined in Sec. IV-E1.

3) Data Statistics: Fig. 2 summarizes the teleoperated corpus collected so far, which with SONIC covers 103 atomic tasks in 11 scene instances and 131 compositional tasks in 13 scene instances across the five scene categories, with a median of 51 episodes per task, amounting to 5,258 atomic episodes (9.3 h) and 6,801 compositional episodes (32.8 h). Of the compositional tasks, 95 chain two steps, 32 chain three, and 4 chain four, and their median episode duration grows with chain length, from 13.6 s for two steps through 21.0 s for three to 34.2 s for four, against 5.8 s for atomic tasks. They also engage the lower body far more often, as the locomotion planner is active in 71% of compositional episodes but in only 7% of atomic ones. Applying the criteria of Fig. 2(c) to every atomic task, a task counts as locomotion when at least half of its episodes walk for at least half of their duration, step, or turn, and as whole-body when at least half of them squat or lean, with whole-body taking precedence. This gives 87 stationary, 9 whole-body, and 7 locomotion tasks. AMO, HEFT, ALMI, and ScaleBFM are each recorded on the same ten bathroom tasks, five atomic and five compositional, which adds 2,101 episodes (9.3 h). Pooled over the five controllers, the corpus holds 103 atomic and 132 compositional tasks, 14,160 episodes, and 51.5 teleoperated hours, which meets the ≈50-hour teleoperation target, and the motion-synthesis pipeline targets a further ≈50 hours. Only episodes passing the shared quality checks contribute to corpus totals. Teleoperated and generated episodes are counted separately and can be matched by episode count for controlled data-source comparisons.

E. Randomization and Real-Robot Validation

Domain randomization uses a composable specification of appearance, camera, and distractor perturbations, separate from the robot and object reset jitter used in collection. Appearance changes affect textures, lighting, material shading, and optionally robot color; camera changes perturb extrinsics and field of view, with optional image-space lens distortion. Distractors are placed near task objects subject to clearance and approach-corridor constraints, with the actual number limited by available space. The *mild* preset combines texture and lighting changes, *standard* adds camera and material-shading perturbations, and *stress* also enables distractors, lens distortion, and robot-color changes. These presets select perturbation combinations whose individual parameters can be overridden; they do not define independent severity scales for visual, geometric, and dynamics randomization. In evaluation, enabled perturbations are drawn using episode-derived seeds, and the resolved specification, seeds, and sampled scene information are recorded with the results. In fixed-scene comparisons, robot and task-object placement is restored to the baseline before applying the configured reset jitter, so inserting distractors does not silently change the task initialization through a different random-number sequence. These perturbations support the seen-task domain-randomization conditions within the main benchmark (Sec. IV-B2); the low-, medium-, and high-difficulty configurations will be recorded explicitly.

Real-robot validation pairs simulated and physical scenes through matched support-surface and object poses, using common task predicates and metrics. The current pilot uses a single manually aligned scene. The image-based reconstruction workflow is designed to reduce manual asset preparation, so Hunyuan3D [58] produces object and fixture meshes with physically-based materials, while collision proxies and inertial parameters are assigned and checked through the same asset-validation process used for open assets. Measured poses then determine the scene layout. Paired trials use matched initial conditions and record the scene specification, pose tolerances, and observed deviations, retaining a non-randomized simulator as a control. Paired evaluation is still incomplete, so the present

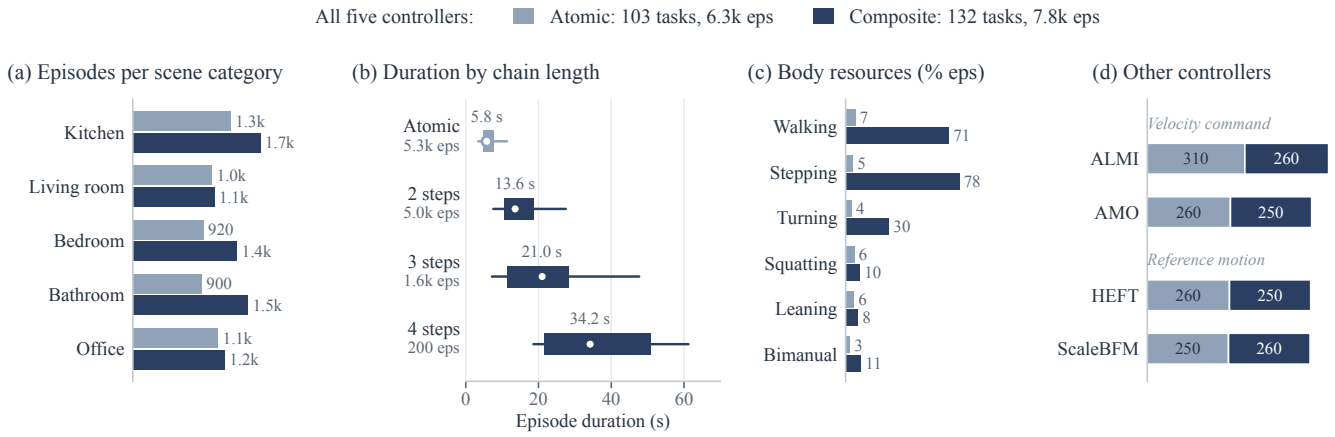


Fig. 2. Statistics of the teleoperated corpus. Totals in the legend pool all five controllers, and episode counts are rounded to two significant figures. Panels (a)–(c) describe the SONIC corpus: (a) episodes per scene category; (b) episode duration by chain length, with the median as a dot labeled with its value, the interquartile range as a bar, and the 5th–95th percentile range as a line; (c) share of episodes that use each body resource, namely walking when the locomotion planner is engaged, stepping when leg-joint travel reaches 25 rad, turning when base yaw changes by at least 45°, squatting when both knees flex by at least 50°, leaning when pelvis tilt reaches 30°, and bimanual grasping. (d) Episodes of the other four controllers, each recorded on the same five atomic and five compositional bathroom tasks and grouped by command interface.

scope is a preliminary hardware feasibility check. Establishing agreement between simulation and real-world policy rankings requires repeated paired evaluations across tasks and scenes.

IV. Experiments

The experimental program is organized around five questions: how policies generalize, how controller choice changes execution, how performance scales with demonstrations, how generated data contributes, and how simulated performance relates to physical deployment. The main benchmark includes both held-out evaluation and domain randomization on seen tasks. The data-generation study contains two complementary parts: downstream learning from different data-source mixtures and direct evaluation of Kimodo variants. Table II summarizes the scope of each study. [Experimental structure confirmed; numerical results and their analysis are pending. All dashes in the result tables denote pending measurements, not zero success or unsupported configurations. Figure and table sizes remain provisional.]

A. Experimental Setup

1) Policy panel and comparison units: The main benchmark trains the complete policy roster in Sec. III-B on 50 atomic and 50 compositional tasks. The multi-controller and data-scale studies each use one representative VLA and one representative world-action model (WAM), ten shared tasks, and all five controllers: SONIC, AMO, HEFT, ALMI, and ScaleBFM. The data-source study uses one VLA and one WAM on twenty tasks. The physical study uses $\pi_{0.5}$, GR00T, and one WAM on three tasks with both simulated and physical evaluation. The representative VLA/WAM selections, exact checkpoint versions, and task manifests will be recorded with the final results. [Proposed default: reuse the representative VLA and WAM across the controlled studies where feasible.]

Each result identifies the policy, controller, training data, and evaluation condition. Controller-specific training preserves the native action and state layouts of Sec. III-B; equal task definitions do not imply interchangeable action arrays. Interface validation and completed evaluation are recorded separately, and a missing or incompatible run is not assigned zero success. [SONIC is the provisional fixed controller for the main benchmark and data-source study; confirm against the final run configurations.] [Finalize controller choices outside the all-controller studies, task IDs, checkpoints, training budgets, seeds, evaluation trial counts, and uncertainty reporting from the actual run records. No numerical values for these settings are fixed by the present reorganization.]

2) Task splits and controlled comparisons: *Seen task* evaluates the training task definitions in their training scene instances, using evaluation initial states. *Unseen task* holds out task definitions, while *unseen scene* evaluates tasks in held-out scene instances. The split manifest must distinguish task definitions from scene instances and exclude held-out evaluation episodes from training. Atomic and compositional success are reported separately in the main benchmark; their aggregation and the number of evaluated tasks accompany each summary. Low-, medium-, and high-DR conditions form an additional evaluation axis on the seen tasks, not a fourth held-out split.

For the controller, data-scale, and source-mixture studies, the proposed comparison protocol holds the evaluation tasks and initial-state seeds fixed within each study. Training budgets and preprocessing are documented so that changes in the factor under study can be distinguished from changes in the training recipe. The number of training seeds, repeated trials, and confidence-interval procedure will be finalized together with the completed evaluation logs.

3) Metrics: [The metric definitions below are retained from the previous draft for continuity and await the separate metric review. Success rate is the primary outcome in the confirmed plan; the controller study uses multiple metrics, and Kimodo receives its own quantitative evaluation.] Every task defines

an ordered set of subgoal predicates, so a rollout is scored at two levels. At the task level, *success rate* is the fraction of rollouts whose terminal goal predicates hold continuously for the task's dwell interval without violating the safety invariants, and *progress* is the fraction of subgoals credited before the first unmet prerequisite, which resolves compositional chains that binary success leaves undifferentiated and cannot be inflated by a later accidental state. At the motion level, *mechanical work* accumulates the effort spent over a rollout and is normalized by base path length, *jerk* summarizes how abruptly the evaluated joints change once every trajectory is resampled to a common rate through a filter fixed before comparison, and *fall rate* is the fraction of rollouts ending with the robot on the ground, counted even where the task predicate had already been satisfied. No single scalar combines the two levels, so task-level metrics quantify capability while motion-level metrics describe execution, and each motion metric is reported both over all valid rollouts and conditioned on success.

B. Main Results: Generalization and Domain Randomization

1) Seen and held-out evaluation: All policies are trained on the same 50 atomic and 50 compositional task definitions and evaluated on seen tasks, unseen tasks, and unseen scenes. Table III separates atomic and compositional success in each condition, with a within-condition summary column. The layout deliberately keeps the three conditions visible rather than collapsing them into a single overall ranking. Completed runs will support analysis of the atomic–compositional gap, transfer to new tasks, and transfer to new scenes; no ranking is inferred before those runs are consolidated.

2) Domain randomization on seen tasks: The same main-benchmark policies are evaluated on seen tasks at low, medium, and high domain-randomization difficulty. Table IV places these conditions beside the corresponding non-DR seen-task result to make the performance change readable. The DR sweep reuses the trained main-benchmark checkpoints. The task manifest and task weights must match across the displayed DR conditions and their baseline. Sec. III-E describes the implemented perturbation families, but the final low/medium/high configurations and their relationship to the existing *mild*, *standard*, and *stress* presets remain to be specified. Preset names alone do not establish a calibrated numerical severity scale. [Record the resolved DR configurations and seeds. Report success at each level; no area-under-curve robustness score is assumed.]

C. Multi-Controller Evaluation

One VLA and one WAM are each trained on ten shared tasks with every controller. The comparison therefore contains two policy families crossed with five controller backends, with training data expressed in each backend's native interface. The task identities are shared across controller conditions, with their scene coverage and atomic/compositional composition specified by the final task manifest. [Fill in the VLA, WAM, ten task IDs, and training budget.]

Table V groups results by policy family and reports success together with execution measures, allowing controller effects to be compared within a policy and then across the two policy families. Work, jerk, and fall rate are retained as candidate columns from the previous draft; their definitions, units, and inclusion await the metric review of Sec. IV-A3. Per-task breakdowns can accompany the aggregate table as supplementary material without expanding the main table into a task-by-metric grid. The analysis will examine capability and execution quality together, without declaring a controller winner from pending values.

D. Data-Scale Study

One VLA and one WAM are trained on ten tasks with all five controllers while varying the number of demonstration episodes per task. Success rate is the response variable. Reusing the ten-task manifest from Sec. IV-C is the proposed default so that the scale study extends the controller comparison directly. The actual episode-count levels and supported range for each controller will follow the available corpus and final run plan.

For each policy–controller pair, nested subsets are proposed to isolate increasing data volume from an unrelated change in episode selection. The training budget, sampling procedure, evaluation conditions, and supported count range will accompany the results. Fig. 3 reserves one panel per policy family, with a curve for each controller, so that curve shape and controller-dependent scaling remain visible without a large multi-dimensional table. Measured points and uncertainty will be added only when the episode counts and evaluations are finalized.

E. Data Generation

This study separates the downstream usefulness of training-data mixtures from the quality of the motion generator itself. The source comparison asks whether a different mixture changes policy success, while the Kimodo comparison combines direct motion visualization with quantitative evaluation.

1) Data-source mixtures: One VLA and one WAM are trained on twenty tasks using different proportions of teleoperated, motion-synthesized, and augmented demonstrations. The complete generation pipeline and source provenance will accompany the finalized corpus. Each mixture is represented explicitly by episode fractions $(r_{\text{tele}}, r_{\text{synth}}, r_{\text{aug}})$, with non-negative fractions that sum to one. The proposed controlled comparison fixes the total episode count per task and evaluation conditions while changing the mixture, separating source composition from the data-volume factor in Sec. IV-D. Exact ratios, total counts, selected models, controller, and evaluation split remain to be finalized.

Table VI uses one row per mixture and paired VLA/WAM success columns. The number of mixture rows can expand when the actual ratios are confirmed; the template does not prescribe particular percentages or source-only runs. If additional evaluation splits are eventually included, each can receive a separate panel without changing the study's role in the paper.

TABLE II
ORGANIZATION OF THE CONFIRMED EXPERIMENTAL PROGRAM. REPRESENTATIVE MODEL IDENTITIES, TASK MANIFESTS, AND RUN-LEVEL SETTINGS REMAIN TO BE FILLED FROM THE FINALIZED EXPERIMENTS.

Study	Models	Training task scope	Comparison axis	Planned presentation
Main benchmark	All policies	50 atomic + 50 compositional	Seen / unseen task / unseen scene; low / medium / high DR on seen tasks	Generalization table + DR table
Multi-controller	One VLA + one WAM	10 shared tasks	All five controllers; multiple metrics	Grouped metric table
Data scale	One VLA + one WAM	10 shared tasks	Demonstrations per task for all five controllers	Two-panel SR curves
Data generation	One VLA + one WAM; three Kimodo variants	20 tasks for source mixtures; Kimodo conditions to specify	Source ratios; pretrained / full fine-tuning / LoRA	Mixture table; motion images + metric table
Real-to-sim and hardware	$\pi_{0.5}$ + GR00T + one WAM	3 paired tasks	Simulation versus physical execution	Paired SR table + scene / rollout images

TABLE III
MAIN BENCHMARK: SUCCESS RATE (%) FOR ALL POLICIES TRAINED ON 50 ATOMIC AND 50 COMPOSITIONAL TASKS. ATOM., COMP., AND AVG. REFER TO ATOMIC TASKS, COMPOSITIONAL TASKS, AND THE WITHIN-CONDITION AGGREGATE; AGGREGATION DETAILS AND EVALUATION COVERAGE WILL ACCOMPANY THE FINAL RESULTS. ALL ENTRIES ARE PENDING.

Policy	Seen task			Unseen task			Unseen scene		
	Atom.	Comp.	Avg.	Atom.	Comp.	Avg.	Atom.	Comp.	Avg.
$\pi_{0.5}$ [26]	–	–	–	–	–	–	–	–	–
XR-1 [51]	–	–	–	–	–	–	–	–	–
GR00T N1.7 [50]	–	–	–	–	–	–	–	–	–
SmolVLA [28]	–	–	–	–	–	–	–	–	–
InternVLA-A1.5 [52]	–	–	–	–	–	–	–	–	–
Ψ_0 [1]	–	–	–	–	–	–	–	–	–
X-VLA [29]	–	–	–	–	–	–	–	–	–
StarVLA-α [48]	–	–	–	–	–	–	–	–	–
PRTS [49]	–	–	–	–	–	–	–	–	–
Diffusion Policy [53]	–	–	–	–	–	–	–	–	–
Fast-WAM [24]	–	–	–	–	–	–	–	–	–
DreamZero [23]	–	–	–	–	–	–	–	–	–

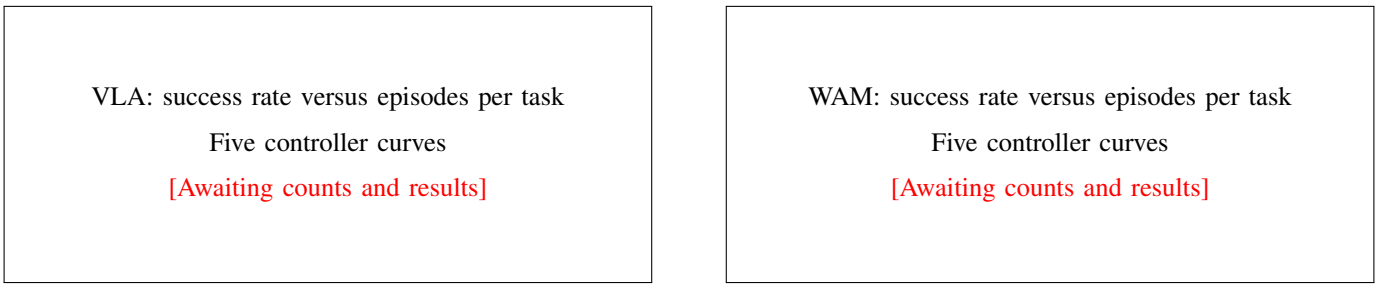


Fig. 3. Proposed presentation of the data-scale study on ten tasks: one panel per policy family, with all five controllers in each panel. The empty panels reserve layout space and contain no illustrative measurements.

2) Kimodo: pretrained, full fine-tuning, and LoRA: We compare the original pretrained Kimodo model, full-parameter fine-tuning, and LoRA fine-tuning. All variants are evaluated with the same frozen SONIC execution controller, consistent with Sec. III-D2. Only the two fine-tuned variants undergo adaptation; the pretrained model serves as the unadapted reference. Matched start/goal conditions and visualization viewpoints are proposed for the three-way comparison. The constraint set, evaluation repeats, fine-tuning budgets, LoRA configuration, and reported metrics will be specified from the finalized runs.

Fig. 4 reserves aligned images for the three variants, and Table VII reserves their quantitative summary. Generated references and executed motions must be labeled distinctly in the visualizations and metric descriptions. Whether full fine-tuning or LoRA performs better remains an empirical result to be reported after the final evaluations.

F. Real-to-Sim and Real-Robot Evaluation

The final study evaluates $\pi_{0.5}$, GR00T, and one WAM on three tasks in both simulation and the physical world. Real-to-sim scene construction provides the paired setting: the physical task arrangement is represented in simulation, with scene correspondence and execution shown alongside the success-rate comparison. Fig. 5 reserves space for matched scene views and rollout frames, and Table VIII pairs simulation and hardware success for each model and task. The specific GR00T version and WAM, task names, controller, trial counts, training-data source, and deployment checkpoints will be recorded with the results.

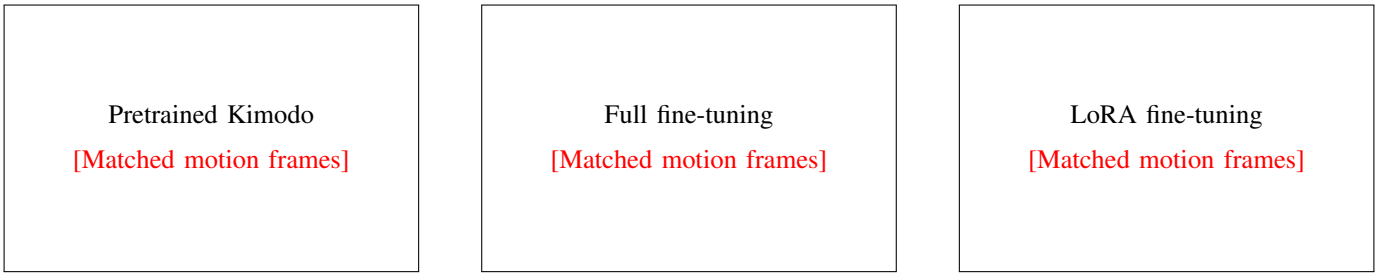


Fig. 4. Planned Kimodo visualization: aligned conditions, timestamps, and viewpoints for the pretrained, fully fine-tuned, and LoRA variants. Final images will distinguish the proposed reference from the frozen controller's executed motion.

TABLE IV
SEEN-TASK SUCCESS RATE (%) UNDER DOMAIN RANDOMIZATION, USING THE MAIN-BENCHMARK POLICY PANEL. THE BASELINE IS THE MATCHING SEEN-TASK CONDITION WITHOUT ADDED DR. DIFFICULTY CONFIGURATIONS AND ALL MEASUREMENTS ARE PENDING.

Policy	Baseline	Low	Medium	High
$\pi_{0.5}$	–	–	–	–
XR-1	–	–	–	–
GR00T N1.7	–	–	–	–
SmolVLA	–	–	–	–
InternVLA-A1.5	–	–	–	–
Ψ_0	–	–	–	–
X-VLA	–	–	–	–
StarVLA-α	–	–	–	–
PRTS	–	–	–	–
Diffusion Policy	–	–	–	–
Fast-WAM	–	–	–	–
DreamZero	–	–	–	–

TABLE V
TWO-POLICY, FIVE-CONTROLLER COMPARISON ON TEN SHARED TASKS. SR IS SUCCESS RATE (%); WORK, JERK, AND FALL ARE PROVISIONAL EXECUTION-METRIC COLUMNS. MODEL NAMES, METRIC DETAILS, AND RESULTS ARE PENDING.

Policy	Controller	SR	Work	Jerk	Fall
VLA	SONIC	–	–	–	–
	AMO	–	–	–	–
	HEFT	–	–	–	–
	ALMI	–	–	–	–
	ScaleBFM	–	–	–	–
WAM	SONIC	–	–	–	–
	AMO	–	–	–	–
	HEFT	–	–	–	–
	ALMI	–	–	–	–
	ScaleBFM	–	–	–	–

The chosen multi-task or single-task training regime and the source of the training demonstrations will be stated explicitly for each reported configuration. For a paired comparison, matched-checkpoint evaluation is preferred where feasible; any environment-specific adaptation must be disclosed so that its effect is not mistaken for a simulation-to-hardware gap. Success rate is the primary outcome, reported separately for each model, task, and execution environment.

V. DISCUSSION AND LIMITATIONS

LOCOMANILAB is intended to make heterogeneous stacks comparable, not to assert that one policy or controller is universally best. Explicit provenance and two-level metrics are essential to that goal.

TABLE VI
DATA-SOURCE MIXTURES ON TWENTY TASKS. RATIO ENTRIES ARE EPISODE FRACTIONS; THE VLA AND WAM COLUMNS REPORT SUCCESS RATE (%). THE DISPLAYED NUMBER OF MIXTURE ROWS IS A LAYOUT PLACEHOLDER. RATIOS, TOTAL EPISODES, AND RESULTS ARE PENDING.

Mixture	Source fractions			SR (%)	
	Teleop.	Synth.	Aug.	VLA	WAM
M1	–	–	–	–	–
M2	–	–	–	–	–
M3	–	–	–	–	–
M4	–	–	–	–	–

TABLE VII
QUANTITATIVE COMPARISON OF THE THREE KIMODO VARIANTS. METRIC A–C RESERVE COLUMN SPACE ONLY; METRIC NAMES, UNITS, CONDITIONS, AND ALL MEASUREMENTS AWAIT CONFIRMATION.

Variant	Metric A	Metric B	Metric C
Pretrained	–	–	–
Full fine-tuning	–	–	–
LoRA fine-tuning	–	–	–

The task suite spans five scene categories with 103 atomic and 132 compositional tasks and is still growing, and two gaps remain. Each task is currently recorded in a single scene instance, so layout variation within a task is not yet covered, and the four controllers other than SONIC cover only ten bathroom tasks. Coverage is bounded by the number of physical robots, integrated controllers, and real-world scenes. Declared interfaces make compatibility explicit but do not eliminate differences in controller feasibility or bandwidth.

Every result here is produced with the policy and the controller running on one machine, so the compute budget is whatever the robot carries. Where that budget binds, inference can instead be organized across device, edge, and cloud [59], [60], and such a deployment would be judged on the inference rate and the delay compensation that the driver of Sec. III-B already makes explicit.

Domain randomization in Sec. IV-B2 evaluates seen tasks under three declared difficulty levels; robustness on those conditions does not by itself establish generalization to unseen tasks or scenes. Mechanical work may rely on a commanded-torque proxy, and jerk is filter-sensitive, so raw signals and filter parameters are released.

TABLE VIII
PAIRED SIMULATION AND REAL-ROBOT SUCCESS RATE (%) ON THREE TASKS. T1–T3, THE EXACT GR00T/WAM VARIANTS, TRAINING REGIME, AND TRIAL COUNTS REMAIN TO BE SPECIFIED. SIM. AND REAL ENTRIES ARE SEPARATE MEASUREMENTS; ALL ARE PENDING.

Policy	Task T1		Task T2		Task T3		Average	
	Sim.	Real	Sim.	Real	Sim.	Real	Sim.	Real
$\pi_{0.5}$	–	–	–	–	–	–	–	–
GR00T	–	–	–	–	–	–	–	–
WAM	–	–	–	–	–	–	–	–

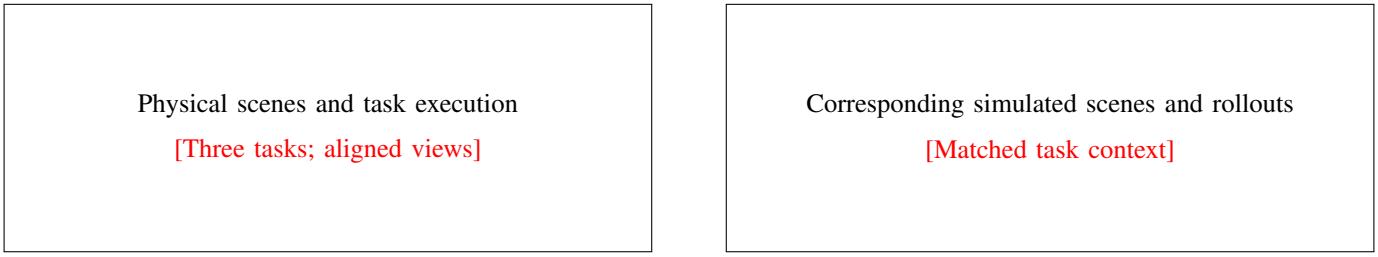


Fig. 5. Planned real-to-sim and hardware visualization. Paired physical and simulated scene views establish task correspondence; selected rollout frames document the evaluated conditions. Scene alignment alone is not evidence of matching policy performance.

None of the above is presented as a novel algorithm. Each component is reported at whatever evidentiary strength it has reached, and omitted from the abstract and contributions where that strength is insufficient.

VI. CONCLUSION

We presented LOCOMANILAB, a benchmark whose unit of evaluation is the complete policy–controller stack, built from a capability-oriented task suite across five scene categories, a versioned contract spanning a broad VLA/WAM panel and five controllers, and a reinforcement-learning motion-synthesis pipeline that complements teleoperation. The experimental program comprises five studies: main results with held-out and domain-randomization evaluation, multi-controller comparison, data scale, data generation, and paired real-to-sim and real-robot evaluation. [Final empirical conclusions will be added after the completed results are consolidated.] We hope the system enables humanoid loco-manipulation results to be compared as complete, reproducible stacks rather than isolated models or demonstrations.

REFERENCES

[1] S. Wei, H. Jing, B. Li, Z. Zhao, J. Mao *et al.*, "Ψ_0: An open foundation model towards universal humanoid loco-manipulation," in *Proceedings of Robotics: Science and Systems*, 2026.
[2] M. Shi, S. Peng, J. Chen, H. Jiang, T. Li, P. Luo, D. Huang, H. Li, and L. Chen, "Unlocking in-the-wild loco-manipulation with robot-free egocentric demonstration," in *Proceedings of Robotics: Science and Systems*, 2026.
[3] Y. Hu, H. Zhu, B. Zheng, Y. Hu, T. Zhang, Z. Chen, J. Zhao, R. Nai, and Y. Gao, "OpenHLM: An empirical recipe for whole-body humanoid loco-manipulation," arXiv:2606.22174, 2026.
[4] H. Jiang, J. Chen, Q. Bu, L. Chen, M. Shi, Y. Zhang, D. Li, C. Suo, C. Wang, Z. Peng, and H. Li, "WholeBodyVLA: Towards unified latent VLA for whole-body loco-manipulation control," in *International Conference on Learning Representations (ICLR)*, 2026.
[5] Z. Luo, Y. Yuan, T. Wang, C. Li, F. Castañeda *et al.*, "SONIC: Supersizing motion tracking for natural humanoid whole-body control," *Science Robotics*, vol. 11, no. 117, p. eaed4592, 2026.
[6] J. Li, X. Cheng, T. Huang, S. Yang, R.-Z. Qiu, and X. Wang, "AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control," in *Proceedings of Robotics: Science and Systems*, 2025.
[7] J. Shi, X. Liu, D. Wang, O. Lu, S. Schwertfeger, C. Zhang, F. Sun, C. Bai, and X. Li, "Adversarial locomotion and motion imitation for humanoid policy learning," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 82 168–82 199.
[8] C. Liu, Q. Lu, G. Yang, X. Shi, C. Yang, Y. Guo, and J. Chen, "HEFT: Heavy-payload full-size humanoid teleoperation with privileged motion guidance and windowed payload curriculum," arXiv:2607.02332, 2026.
[9] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel, "Humanoid-Bench: Simulated humanoid benchmark for whole-body locomotion and manipulation," in *Proceedings of Robotics: Science and Systems*, 2024.
[10] S. Wei, Z. Ni, J. Liu, Z. Zhao, J. Ye *et al.*, "SIMPLE: Simulation-based policy learning and evaluation for humanoid loco-manipulation," arXiv:2606.08278, 2026.
[11] K. Lin, A. Mandlekar, C. R. Garrett, N. Chernyadev, Y. Fang, R. Ding, Y. Xie, J. Tran, L. Fan, and Y. Zhu, "HumanoidMimicGen: Data generation for loco-manipulation via whole-body planning," arXiv:2605.27724, 2026.
[12] Y.-J. Wang, J. Li, S. Chen, T. E. Truong, P. Xu, P. Abbeel, R. Duan, K. Sreenath, A. Kanazawa, C. Sferrazza, G. Shi, and C. K. Liu, "VLK: Learning humanoid loco-manipulation from synthetic interactions in reconstructed scenes," arXiv:2606.30645, 2026.
[13] T. Wang, Z. Xie, B. Yang, Y. Wang, Z. Yuan *et al.*, "HumanoidArena: Benchmarking egocentric hierarchical whole-body learning," arXiv:2606.17833, 2026.
[14] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. M. Kitani, C. Liu, and G. Shi, "OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 1516–1540.
[15] T. He, W. Xiao, T. Lin, Z. Luo, Z. Xu, Z. Jiang, J. Kautz, C. Liu, G. Shi, X. Wang, L. J. Fan, and Y. Zhu, "HOVER: Versatile neural whole-body controller for humanoid robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 9989–9996.
[16] Z. Chen, M. Ji, X. Cheng, X. Peng, X. B. Peng, and X. Wang, "GMT: General motion tracking for humanoid whole-body control," arXiv:2506.14770, 2025.
[17] J. Han, W. Xie, J. Zheng, J. Shi, W. Zhang, T. Xiao, and C. Bai, "KungfuBot2: Learning versatile motion skills for humanoid whole-body control," arXiv:2509.16638, 2025.
[18] W. Zeng, K. Yin, X. Niu, S. Lu, W. Zhong, J. Chen, F. Jia, X. Chen, Z. Wang, F. Xu, M. Zhou, K. Li, W. Zhang, H. Wang, L. Yi, D. Lin, J. Pang, and J. Wang, "Scaling behavior foundation model for humanoid robots," arXiv:2607.15163, 2026.
[19] M. Yuan, T. Yu, W. Ge, X. Yao, D. Li, H. Wang, J. Chen, B. Li, W. Zhang, W. Zeng, H. Chen, and X. Jin, "A survey of behavior foundation model: Next-generation whole-body control system of humanoid

robots," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 4, pp. 4909–4927, 2026.
[20] H. Weng, Y. Li, N. Sobanbabu, Z. Wang, Z. Luo, T. He, D. Ramanan, and G. Shi, "HDMI: Learning interactive humanoid whole-body control from human videos," arXiv:2509.16757, 2025.
[21] H. Wang, W. Zhang, R. Yu, T. Huang, J. Ren, F. Jia, Z. Wang, X. Niu, X. Chen, J. Chen, Q. Chen, J. Wang, and J. Pang, "PhysHSI: Towards a real-world generalizable and natural humanoid-scene interaction system," arXiv:2510.11072, 2025.
[22] Y. Lin, J. Shi, D. Wang, J. Kong, Y. Liu, C. Bai, and X. Li, "Pro-HOI: Perceptive root-guided humanoid-object interaction," arXiv:2603.01126, 2026.
[23] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu *et al.*, "World action models are zero-shot policies," arXiv:2602.15922, 2026.
[24] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, "Fast-WAM: Do world action models need test-time future imagination?" arXiv:2603.16666, 2026.
[25] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, L. Zhang, M. Yu, Z. Gao, N. Xue, B. Zhou, X. Zhu, M. Ding, Y. Shen, and Y. Xu, "Causal world modeling for robot control," in *Proceedings of Robotics: Science and Systems*, 2026.
[26] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 17–40.
[27] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da *et al.*, "GR00T N1: An open foundation model for generalist humanoid robots," arXiv:2503.14734, 2025.
[28] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma *et al.*, "SmolVLA: A vision-language-action model for affordable and efficient robotics," arXiv:2506.01844, 2025.
[29] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang *et al.*, "X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," in *International Conference on Learning Representations (ICLR)*, 2026.
[30] X. Chen, Y. Chen, Y. Fu, N. Gao, J. Jia *et al.*, "InternVLA-M1: A spatially guided vision-language-action framework for generalist robot policy," arXiv:2510.13778, 2025.
[31] J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5625–5644, 2024.
[32] H. Zhou, R. Wang, Y. Tai, Y. Deng, G. Liu, and K. Jia, "YOTO++: Learning long-horizon closed-loop bimanual manipulation from one-shot human video demonstrations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 48, no. 9, pp. 10 695–10 712, 2026.
[33] Z. Zhao, Y. Zhao, G. Zhang, C. Liu, M. Zheng, and W. Lian, "HALOMI: Learning humanoid loco-manipulation with active perception from human demonstrations," arXiv:2606.18772, 2026.
[34] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, "RLBench: The robot learning benchmark & learning environment," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 3019–3026, 2020.
[35] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 776–44 791.
[36] Y. Chen, Y. Geng, F. Zhong, J. Ji, J. Jiang, Z. Lu, H. Dong, and Y. Yang, "Bi-DexHands: Towards human-level bimanual dexterous manipulation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 2804–2818, 2024.
[37] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, "RoboCasa: Large-scale simulation of household tasks for generalist robots," in *Proceedings of Robotics: Science and Systems*, 2024.
[38] S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T.-K. Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su, "Demonstrating GPU parallelized robot simulation and rendering for generalizable embodied AI with ManiSkill3," in *Proceedings of Robotics: Science and Systems*, 2025.
[39] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu, "RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots," in *International Conference on Learning Representations (ICLR)*, 2026.
[40] Open X-Embodiment Collaboration, "Open X-embodiment: Robotic learning datasets and RT-X models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6892–6903.
[41] X. Li, K. Hsu, J. Gu, O. Mees, K. Pertsch, H. R. Walke *et al.*, "Evaluating real-world robot manipulation policies in simulation," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 3705–3728.
[42] P. Guruprasad, H. Sikka, J. Song, Y. Wang, and P. P. Liang, "Benchmarking vision, language, & action models on robotic learning tasks," arXiv:2411.05821, 2024.
[43] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
[44] M. Deitke, D. Schwenk, J. Salvador, L. Weihs, O. Michel, E. VanderBilt, L. Schmidt, K. Ehsani, A. Kembhavi, and A. Farhadi, "Objaverse: A universe of annotated 3D objects," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 13 142–13 153.
[45] Y. Kim, W. Pumacay, O. Rayyan, M. Argus, W. Han, E. Vanderbilt, J. Salvador, A. Deshpande, R. Hendrix, S. Jauhri, S. Liu, N. M. M. Shafiullah, M. Guru, A. Eftekhar, K. Farley, D. Clay, J. Duan, A. Guru, P. Wolters, A. Herrasti, Y.-C. Lee, G. Chalvatzaki, Y. Cui, A. Farhadi, D. Fox, and R. Krishna, "MolmoSpaces: Large-scale open ecosystem for robot manipulation and navigation," in *Proceedings of Robotics: Science and Systems*, 2026.
[46] Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, C. Callison-Burch, M. Yatskar, A. Kembhavi, and C. Clark, "Holodeck: Language guided generation of 3D embodied AI environments," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 277–16 287.
[47] StarVLA Community, "StarVLA: A lego-like codebase for vision-language-action model developing," arXiv:2604.05014, 2026.
[48] J. Ye, N. Gao, S. Yang, J. Zheng, Z. Wang *et al.*, "StarVLA-α: Reducing complexity in vision-language-action systems," arXiv:2604.11757, 2026.
[49] Y. Zhang, J. Zhao, C. Fan, F. Yan, T. Li *et al.*, "PRTS: A primitive reasoning and tasking system via contrastive representations," arXiv:2604.27472, 2026.
[50] NVIDIA, "NVIDIA Isaac GR00T N1.7-3B model card," Hugging Face, 2026, accessed: Sep. 28, 2026. [Online]. Available: https://huggingface.co/nvidia/GR00T-N1.7-3B
[51] S. Fan, K. Wu, Z. Che, X. Wang, D. Wu, F. Liao, N. Liu *et al.*, "XR-1: Towards versatile vision-language-action models via learning unified vision-motion representations," arXiv:2511.02776, 2025.
[52] H. Ma, J. Cai, X. Xu, H. Li, Y. Yang, Y. Tian, J. Cao *et al.*, "InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization," arXiv:2607.04988, 2026.
[53] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
[54] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, Y. Jiang, T. Wang, U. Iqbal, D. Minor, M. de Ruyter, J. Li, C. Tessler, E. Lim, E. Jeong, S. Wu, E. Hassani, M. Huang, J.-B. Yu, C. Chung, L. Song, O. Dionne, J. Kautz, S. Yuen, and S. Fidler, "Kimodo: Scaling controllable human motion generation," arXiv:2603.15546, 2026.
[55] M. Zhang, Z. Cai, L. Pan, F. Hong, X. Guo, L. Yang, and Z. Liu, "MotionDiffuse: Text-driven human motion generation with diffusion model," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 6, pp. 4115–4128, 2024.
[56] W. Zhu, X. Ma, D. Ro, H. Ci, J. Zhang, J. Shi, F. Gao, Q. Tian, and Y. Wang, "Human motion generation: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 4, pp. 2430–2449, 2024.
[57] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, N. Ratliff, and D. Fox, "cuRobo: Parallelized collision-free minimum-jerk robot motion generation," arXiv:2310.17274, 2023.
[58] Team Hunyuan3D, S. Yang, M. Yang, Y. Feng, X. Huang, S. Zhang *et al.*, "Hunyuan3D 2.1: From images to high-fidelity 3D assets with production-ready PBR material," arXiv:2506.15442, 2025.
[59] J. Shao and X. Li, "AI Flow at the network edge," *IEEE Network*, vol. 40, no. 1, pp. 330–336, 2026.
[60] H. An, W. Hu, S. Huang, S. Huang, R. Li, Y. Liang, J. Shao, Y. Song, Z. Wang, C. Yuan *et al.*, "AI Flow: Perspectives, scenarios, and approaches," *Vicinagearth*, vol. 3, no. 1, p. 1, 2026.